Confidential Inference PoC on an Emulated ARM Device

A step-by-step guide to setting up OP-TEE/TrustZone under QEMU (no physical Raspberry Pi 4 required), with limitations and mitigations at every stage, for your RawTFNet deepfake-detection edge gateway thesis.

---

0. Read this first: what QEMU actually emulates

You said you want to use a “Raspberry Pi emulator.” It's important to get this precise before you build anything, because it changes what you can honestly write in your thesis.

There is no mainstream, official emulator that boots “a Raspberry Pi 4” with working TrustZone/OP-TEE support inside QEMU. QEMU has a Raspberry Pi 3/4 machine model, but Broadcom never published the secure-boot and TrustZone configuration details for the BCM2711 (Pi 4) SoC, so OP-TEE has no official, maintained port to the real Raspberry Pi 4 board — emulated or physical.

What OP-TEE officially supports and what you should use instead is the **QEMU "virt" reference platform (qemu_v8, Armv8-A)**: a generic ARMv8 board with a proper TrustZone-capable CPU model, secure/non-secure worlds, and full OP-TEE support maintained directly by the OP-TEE project. This is the standard platform almost every OP-TEE research paper and tutorial actually uses when they say “we evaluated on QEMU.”

LIMITATION	Your thesis proposal says “Raspberry Pi 4” and “QEMU emulator” in the same sentence, but the two aren't literally compatible for TrustZone work.
ANSWER	Frame it accurately: your target deployment platform is the Raspberry Pi 4 (ARMv8-A, matches the Pi 4's Cortex-A72), and for the confidential-inference PoC you demonstrate feasibility on the QEMU ARMv8-A “virt” reference platform, which uses the same CPU architecture and the same TrustZone security model. This is a completely standard and defensible framing — state it explicitly in your methodology so it reads as a deliberate scope decision, not a gap.

1. Prerequisites

Do this on a native Linux machine or a Linux VM with virtualization enabled — not Git Bash/MINGW64 on Windows.

LIMITATION	Your current dev setup is Windows + Git Bash (MINGW64). OP-TEE's build system (repo tool, Buildroot, Make, device tree compiler, cross toolchains) is Linux-only and will not build correctly under WSL1, Cygwin, or MINGW.

ANSWER	Use Ubuntu 22.04 or 24.04, either as: (a) a native dual-boot/second machine, (b) WSL2 (not WSL1) on your Windows machine, or (c) a VirtualBox/VMware Ubuntu VM with 4+ CPU cores, 8+ GB RAM, and ~60 GB free disk. WSL2 is the least disruptive option since you keep using Windows for everything else.

Install build dependencies (Ubuntu 22.04/24.04):

```
sudo apt update && sudo apt install -y android-tools-adb android-tools-fastboot \
autoconf automake bc bison build-essential ccache cscope curl
device-tree-compiler \
expect flex ftp-upload gdisk libattr1-dev libcap-dev libcap-ng-dev libfdt-dev \
libftdi-dev libglib2.0-dev libhidapi-dev libncurses5-dev libpixman-1-dev \
libslirp-dev libssl-dev libtool make mtools netcat ninja-build \
python3-cryptography python3-pip python3-pyelftools python3-serial \
python3-venv rsync unzip uuid-dev xdg-user-dirs xterm xz-utils zlib1g-dev git
```

Install the **repo** tool (Google's multi-repo manager — OP-TEE uses it to pull ~15 git repos in sync):

```
mkdir -p ~/bin
curl https://storage.googleapis.com/git-repo-downloads/repo > ~/bin/repo
chmod a+x ~/bin/repo
export PATH=~/bin:$PATH # add this line to ~/.bashrc too
git config --global user.name "Your Name"
git config --global user.email "you@example.com"
```

LIMITATION	The full source checkout (Linux kernel, OP-TEE OS, Buildroot, QEMU, TF-A, toolchains) is large and the first build can take 1–3+ hours depending on your machine and connection.
ANSWER	Budget a half-day for the very first build. Subsequent rebuilds after code changes take seconds to minutes because only the changed component recompiles. Run it once, let it finish, then treat the resulting tree as a stable base you branch experiments from.

2. Fetch and build OP-TEE for QEMU v8

From inside your Ubuntu environment:

```
mkdir -p ~/optee && cd ~/optee
repo init -u https://github.com/OP-TEE/manifest.git -m qemu_v8.xml
repo sync -j4
cd build
make toolchains -j2
make -j$(nproc)
```

This builds: OP-TEE OS (secure world), TF-A (ARM Trusted Firmware, does the secure boot handoff), a Linux kernel + Buildroot rootfs (normal world), OP-TEE client tools, and a matching QEMU binary. The **qemu_v8.xml** manifest is the ARMv8-A configuration — use this one, not the v7 (default.xml) manifest.

LIMITATION	"make" with no target tries to both build and flash to real hardware for non-QEMU platforms, and the official docs assume you already know which manifest maps to which board.

ANSWER	For QEMU, plain `make` and `make run` are correct — there's no flashing step because QEMU boots directly from the build output directory. If a step fails, re-run with `make 2>&1 | tee build.log` and grep the log; 90% of first-time failures are a missing apt package from the prerequisites list above.

3. Boot it and confirm the secure/normal world split works

```
cd ~/optee/build
make run
```

This opens the QEMU monitor plus two xterm windows: one is the **normal world** console (a regular Linux shell, Buildroot-based), the other is the **secure world** console (OP-TEE's own logging). At the QEMU monitor prompt type `c` and Enter to continue booting. Log in to the normal-world console as `root` (no password), then run the built-in test suite:

```
xtest
```

If xtest passes, you have a working TrustZone split: untrusted Linux in the normal world, and an isolated trusted OS (OP-TEE) in the secure world, talking to each other over the standard TEE Client API. This is your confidential-inference substrate.

LIMITATION	No physical hardware root of trust exists in this setup — QEMU's "secure world" is a software CPU-mode partition, not a real silicon TrustZone with fused keys, secure boot chain, or tamper resistance.
ANSWER	This is expected and fine for a feasibility PoC. State explicitly in your thesis that QEMU validates the software architecture and isolation model (TA/CA separation, confidential inference path, attack-surface reduction) but does not provide hardware-backed attestation — that would require physical Pi 4 -class silicon with a real TrustZone implementation, which is out of scope for the PoC and can be named as future work.

4. Understand the TA / CA model before writing your own

Everything in OP-TEE is split into two pieces that you write separately:

- **TA (Trusted Application)** — code that runs inside the secure world. This is where your model inference should eventually happen. Written in C, compiled against optee_os's TA devkit, signed, and loaded on demand.
- **CA (Client Application)** — an ordinary Linux program in the normal world (this is where your Asterisk rolling-audio-buffer glue lives). It opens a session with the TA and invokes commands through the GlobalPlatform TEE Client API (`libteec`).

Build and run the official hello_world example first — don't skip this:

```
cd ~/optee
repo sync -j4 # pulls optee_examples if not already present
cd build
make -j$(nproc) # rebuilds rootfs with examples included
make run
```

```
# inside the normal-world console:
optee_example_hello_world
```

LIMITATION	The official examples repo (optee_examples) targets basic arithmetic/AES demos, nothing close to ML inference, so there is no template to copy directly for your use case.
ANSWER	That's expected — hello_world's only purpose is to prove your toolchain, signing, and CA↔TA session handshake all work end-to-end before you add any complexity. Once it runs cleanly, you clone its folder structure (Makefiles, UUID generation, host/ and ta/ subfolders) as the skeleton for your own TA.

5. Getting RawTFNet inference to run inside the secure world (the hard part)

This is where “simplified PoC” matters most. Full C/C++ deep-learning inference inside a TA is a real engineering project on its own, so here is the realistic, scoped path:

Step 5.1 — Export the trained model to a minimal-runtime format

```
# from your existing RawTFNet-Pytorch repo, CPU-only export
python export_model.py \
--checkpoint Best_RawTFNet_16.pth \
--format onnx \
--output rawtfnet.onnx
```

Prefer exporting to ONNX and then to a fixed-graph / static-shape format if possible. TAs have tiny, fixed secure-memory heaps (a few MB by default), so anything requiring a full dynamic ML framework will not fit or build inside the TA devkit.

LIMITATION	PyTorch, ONNXRuntime, and TensorFlow all assume a POSIX-ish environment with dynamic memory, threads, and large heaps — none of that exists inside a TA, which is a minimal, statically-linked, single-threaded, restricted-libc environment.
ANSWER	Do not try to port PyTorch/ONNXRuntime into the TA. Instead use one of: (a) **microTVM / TVM** to compile the exported graph down to standalone, dependency-free C, (b) hand-written C inference for just the convolution/GRU layers RawTFNet uses (feasible since it's a fairly small model), or (c) a minimal fixed-point inference engine such as CMSIS-NN style kernels if you quantize. Given your timeline, (a) TVM's 'ahead-of-time C codegen' is the best ratio of effort to credibility for a thesis PoC.

Step 5.2 — Build a minimal TA that runs one forward pass

Scope the TA to a single command: `TA_CMD_RUN_INFERENCE`, taking a fixed-length float buffer (a windowed audio feature vector) in, and returning a spoof/bonafide score out. Keep the model weights as a static const C array compiled directly into the TA binary for the PoC — don't build a secure filesystem / key-provisioning flow yet.

LIMITATION	Compiled-in weights mean the TA binary itself becomes something you'd need to re-sign every time the model is retrained — not workable for your longer-term federated-learning goal.
ANSWER	For the PoC this is fine and worth stating as a deliberate simplification. Note in your thesis that a production version would load weights via OP-TEE's secure storage API (encrypted, sealed to the device) so the TA binary stays fixed while only the weights update — this is exactly the piece you're already correctly deferring alongside full federated learning.

Step 5.3 — Write the CA that feeds it from your rolling audio buffer

The CA lives in the normal world next to your Asterisk integration. Its job is narrow: pull the next audio window from the rolling buffer, extract the same features RawTFNet expects, open a TEE session, invoke `TA_CMD_RUN_INFERENCE`, and read back the score.

```
// host/main.c (sketch)
TEEC_Context ctx; TEEC_Session sess;
TEEC_InitializeContext(NULL, &ctx);
TEEC_OpenSession(&ctx, &sess, &ta_uuid, TEEC_LOGIN_PUBLIC, NULL, NULL, NULL);

TEEC_Operation op = {0};
op.paramTypes = TEEC_PARAM_TYPES(TEEC_MEMREF_TEMP_INPUT, TEEC_VALUE_OUTPUT,
TEEC_NONE, TEEC_NONE);
op.params[0].tmpref.buffer = feature_window;
op.params[0].tmpref.size = sizeof(feature_window);

TEEC_InvokeCommand(&sess, TA_CMD_RUN_INFERENCE, &op, NULL);
float score = op.params[1].value.a;
```

LIMITATION	Feature extraction (framing, windowing, whatever front-end RawTFNet uses) still runs in the untrusted normal world in this design, so an attacker with normal-world root could feed the TA manipulated features.
ANSWER	Acceptable and standard for a PoC — name it explicitly as a known boundary in your threat model. The property you ARE demonstrating is that the model weights and the inference computation itself are opaque to and untamperable by normal-world code, which is the core confidential-inference claim. Fully moving feature extraction into the TA is a valid 'future work' item.

6. Recommended order of work for your PoC

Stage	Goal	Why it's ordered here
M1	QEMU v8 OP-TEE boots, xtest passes	Confirms toolchain + environment
M2	hello_world TA/CA runs successfully	Confirms your build/sign/invoke pipeline
M3	Export RawTFNet to ONNX, compile via TVM AoT to standalone C	De-risked outside the TA first, on your host

M4	Link the generated C inference code into a new TA skeleton (copied from hello_world)	First real secure-world build
M5	CA sends one static test vector, TA returns a score matching your PyTorch reference output	Correctness check, still offline
M6	CA reads from the Asterisk rolling buffer and drives the TA in a loop	End-to-end live PoC

Each milestone is independently demonstrable and citable in your thesis write-up even if you run out of time before M6 — which matters for how you scope your final chapters.

7. Full limitations summary (for your thesis ‘Limitations’ section)

Limitation	Detail
No physical device	Everything validated on QEMU's generic ARMv8 virt board, not real Pi 4 silicon or TrustZone hardware.
No hardware root of trust	QEMU's secure world is software-only; no fused keys, no real secure boot chain, no tamper resistance.
No ML framework in the TA	PyTorch/ONNXRuntime cannot run inside OP-TEE; inference must be pre-compiled to dependency-free C (e.g. via TVM AoT) or hand-written.
Static compiled-in weights	PoC skips secure storage / key provisioning for weights, deferred alongside federated learning.
Feature extraction stays untrusted	Only the model forward pass is confidential in this scope; front-end audio processing runs in the normal world.
Single-threaded, tiny heap in TA	Secure-world memory is small by default (a few MB); large models or batch inference are not realistic without reconfiguring TA heap size, which itself narrows the security margin.
Timing / side channels out of scope	No side-channel hardening (cache-timing, power analysis) is attempted — reasonable to state explicitly as future work.

8. Quick reference: essential commands

```
# one-time setup
mkdir -p ~/optee && cd ~/optee
repo init -u https://github.com/OP-TEE/manifest.git -m qemu_v8.xml
repo sync -j4
cd build && make toolchains -j2 && make -j$(nproc)

# every time you want to boot it
cd ~/optee/build && make run
# at QEMU monitor prompt: c (then Enter)
# log in as: root (no password)
xtest # sanity check
optee_example_hello_world # confirms TA/CA pipeline
```

Source for the build/run steps above: the official OP-TEE documentation (optee.readthedocs.io, ‘Building → QEMU v8’ and ‘Building → build git’ pages) and the OP-TEE/build repository's qemu_v8 manifest.